

- Project_Refactor 中 BNO085 偏航角读取链路详解
 - 1. 文档目的
 - 2. 整体结论先讲清楚
 - 3. 硬件接入与引脚定义
 - 3.1 I2C 引脚
 - 3.2 BNO085 额外控制引脚
 - 3.3 支持探测的 I2C 地址
 - 4. 为什么文件名还是 ICM42688, 但实际上已经在读 BNO085
 - 4.1 结构体名字仍然沿用旧名
 - 4.2 对外接口名也仍然是旧名
 - 4.3 但实现内容已经是 BNO085 主路径
 - 5. 上电后的初始化入口在哪里
 - 5.1 main() 是总入口
 - 5.2 Control_Init() 是 IMU 初始化的直接入口
 - 6. ICM42688_Init() 实际做了什么
 - 6.1 配置 GPIO 与软件 I2C
 - 6.2 清空运行时状态
 - 6.3 轮流尝试多个地址
 - 6.4 如果没成功, 再扫描总线
 - 6.5 如果仍失败, 再带硬复位重试一轮
 - 6.6 成功后退出初始化忙状态
 - 7. BNO085 初始化内部的关键握手流程
 - 7.1 确认设备存在
 - 7.2 请求 Product ID
 - 7.3 开启需要的传感器报告
 - 7.4 等待第一帧姿态报告
 - 7.5 初始化成功的最终标志
 - 8. 软件 I2C 与 SHTP 收发是怎么工作的
 - 8.1 软件 I2C
 - 8.2 BNO085 的收发不是寄存器读写, 而是包式协议
 - 8.3 INT 脚参与了读包门控
 - 9. 收到包之后, 如何从四元数算出 yaw
 - 9.1 解析姿态报告
 - 9.2 识别 report id
 - 9.3 从 report 中取出四元数
 - 9.4 四元数转换成欧拉角
 - 10. 为什么 ICM42688_UpdateYaw() 还要再更新一次 yaw

- 10.1 BNO085_UpdateEuler() 更像 “原始姿态解算结果落表”
- 10.2 ICM42688_UpdateYaw() 做的是运行时保护与动态量计算
- 11. 主循环中，yaw 是在什么时候持续刷新的
 - 11.1 Control_Tick() 每 1ms 走一次主逻辑节拍
 - 11.2 IMU 更新不是每次都读，而是每 5ms 读一次
 - 11.3 control_imu_update() 是 yaw 刷新的主入口
- 12. yaw 是如何进入控制环的
 - 12.1 锁定目标航向
 - 12.2 启动时重新建立 yaw 零点
 - 12.3 计算偏航误差
 - 12.4 生成 heading correction
 - 12.5 混入左右轮输出
- 13. 工程是靠什么判定 “已经成功读到偏航角” 的
 - 13.1 第一层：设备已识别
 - 13.2 第二层：初始化阶段已进入 READY
 - 13.3 第三层：姿态样本已建立
 - 13.4 第四层：运行期 yaw 在刷新
- 14. 当前工程有哪些现成调试手段可以验证 yaw 读取成功
 - 14.1 OLED 主界面
 - 14.2 OLED 引脚/ACK 诊断页
 - 14.3 VOFA 文本调试输出
 - 14.4 串口命令 #IMU_INIT!
- 15. 运行期失败后，工程是如何自动恢复的
 - 15.1 未 ready 长时间失败，会重初始化
 - 15.2 已 ready 但运行期卡死，也会重初始化
 - 15.3 恢复时会把姿态状态清零
- 16. 为什么这个工程最终能够 “成功读到 yaw”
 - 16.1 已经不再把 BNO085 当成寄存器型 IMU 使用
 - 16.2 初始化过程不是单点硬编码，而是有多轮兜底
 - 16.3 收包时利用了 INT 做数据门控
 - 16.4 不是只拿四元数，还做了姿态后处理
 - 16.5 启动流程会重新锁定当前朝向
- 17. 一句话总结这条 yaw 链路
- 18. 简化版调用链总览
 - 18.1 上电初始化链路
 - 18.2 运行期读取链路
 - 18.3 控制链路
- 19. 对后续阅读源码的建议

Project_Refactor 中 BNO085 偏航角读取链路详解

1. 文档目的

本文详细说明 `Project_Refactor` 工程中，系统是如何从 `BNO085` 读到偏航角 `yaw` 的，以及这条链路为什么能够成功工作。

重点回答以下问题：

- 当前工程里究竟是谁在负责读 IMU
- 为什么文件名还叫 `ICM42688`，但实际上已经在读 `BNO085`
- `BNO085` 是通过什么引脚、什么协议、什么初始化流程接入的
- `yaw` 是如何从原始数据一步步变成控制系统里的 `sys->icm.yaw`
- `yaw` 后续又是如何参与航向锁定、误差计算和差速控制的
- 系统是靠什么判据判断“已经成功读到偏航角”的
- 出现异常时，当前工程又是如何恢复的

本文对应的核心文件如下：

- `User/main.c`
- `Hardware/Control.c`
- `Hardware/ICM42688.c`
- `Hardware/ICM42688.h`
- `Hardware/BNO085_DebugSerial.c`

需要特别说明的是：

- 虽然文件名仍然叫 `ICM42688.c/.h`
- 但在当前 `Project_Refactor` 版本中，这组文件实际已经承担了 **BNO085 的初始化、握手、收包、解析和 yaw 输出** 工作

也就是说，名字还是旧名字，但实现内容已经以 `BNO085` 为主。

2. 整体结论先讲清楚

Project_Refactor 读 yaw 的完整主链路可以概括成下面这 8 步：

1. `main()` 调用 `Control_Init(&g_controlSys, 0)`
2. `Control_Init()` 内部调用 `ICM42688_Init()`
3. `ICM42688_Init()` 实际配置的是 BNO085 所需引脚，并按多个 I2C 地址尝试初始化
4. 初始化成功后，驱动通过 SHTP 协议向 BNO085 请求 Product ID，并开启 Rotation Vector / Game Rotation Vector / Gyroscope / Accelerometer 报告
5. 后续主循环里，`Control_Tick()` 每 5ms 调用一次 `control_imu_update()`
6. `control_imu_update()` 调用 `ICM42688_ReadAll()`；该函数实际会从 BNO085 收包并解析姿态报告
7. 一旦收到了 Rotation Vector 或 Game Rotation Vector，驱动就会把四元数 `q0/q1/q2/q3` 转成欧拉角，其中 yaw 会写入 `sys->icm.yaw`
8. `Control.c` 再用 `targetYaw - currentYaw` 计算 `yawErr`，生成 `headingCorr`，并把修正量混到左右轮输出中

所以当前工程中，yaw 不是 MPU6050/ICM42688 那种“本地自己做姿态解算”的主路径，而是：

- 由 BNO085 自己输出融合后的姿态四元数
- STM32 侧再做一层 四元数 -> yaw/pitch/roll 的转换和运行期修正

3. 硬件接入与引脚定义

在 `Hardware/ICM42688.c` 顶部，当前工程对 BNO085 的接线定义如下：

3.1 I2C 引脚

- SCL -> PB12
- SDA -> PB13

对应宏定义：

- ICM_I2C_SCL_PIN = GPIO_Pin_12
- ICM_I2C_SDA_PIN = GPIO_Pin_13
- ICM_I2C_PORT = GPIOB

3.2 BNO085 额外控制引脚

- RST -> PB14
- INT -> PA15

对应宏定义：

- BNO085_RESET_PIN = GPIO_Pin_14
- BNO085_RESET_PORT = GPIOB
- BNO085_INT_PIN = GPIO_Pin_15
- BNO085_INT_PORT = GPIOA

3.3 支持探测的 I2C 地址

驱动会依次尝试以下地址：

- 0x4A
- 0x4B
- 0x28
- 0x29

对应定义：

- BNO085_ADDR_ALT = 0x4A
- BNO085_ADDR_DEFAULT = 0x4B
- BNO085_ADDR_DOC_DEFAULT = 0x28
- BNO085_ADDR_DOC_ALT = 0x29

这里同时兼容了：

- 常见板级资料中出现的 0x4A/0x4B
- 某些文档或工具里会写成的 0x28/0x29

因此，这个工程在地址兼容性上做得比较宽松。

4. 为什么文件名还是 ICM42688，但实际上已经在读 BNO085

这是当前工程里最容易让人误解的一点。

4.1 结构体名字仍然沿用旧名

在 `Hardware/ICM42688.h` 中，姿态数据结构体名仍然是：

- `ICM42688_Data_t`

这个结构体里保存了：

- 加速度
- 角速度
- `yaw / pitch / roll`
- `q0 / q1 / q2 / q3`
- `yawRate`
- `ahrsInitd`
- `yawSampleUpdated`

这说明从控制系统视角看，IMU 数据统一装在这一份结构里，不管底层真实硬件已经换成了谁。

4.2 对外接口名也仍然是旧名

例如：

- `ICM42688_Init()`
- `ICM42688_ReadAll()`
- `ICM42688_ResetAttitude()`
- `ICM42688_UpdateYaw()`
- `ICM42688_GetYawError()`

这些名字都保留了旧接口形式。

4.3 但实现内容已经是 BNO085 主路径

在 `Hardware/ICM42688.c` 内部，已经明确存在大量 `BNO085_*` 函数，例如：

- `BNO085_DevicePresent()`
- `BNO085_HardwareReset()`
- `BNO085_RequestProductId()`
- `BNO085_EnableReport()`
- `BNO085_ReceivePacket()`
- `BNO085_ParseSensorReport()`
- `BNO085_UpdateEuler()`

所以可以把它理解成：

- 外壳接口仍叫 `ICM42688`
- 内核实现已经切换成 `BNO085`

这样做的好处是：

- 上层控制代码改动少
- `Control.c` 不需要大规模改接口名
- 旧代码框架能平滑迁移到新 IMU

5. 上电后的初始化入口在哪里

5.1 `main()` 是总入口

在 `User/main.c` 中：

- `main()` 启动后先初始化 OLED、按键等
- 然后调用 `ICM42688_DiagPinsInit()` 做诊断引脚初始化
- 再调用 `Control_Init(&g_controlSys, 0)` 初始化控制系统

这里第二个参数 `0` 表示：

- 不跳过 IMU 初始化
- 即本次要以正常模式启动 IMU

5.2 `Control_Init()` 是 IMU 初始化的直接入口

在 `Hardware/Control.c` 中, `Control_Init()` 做了这些事:

- 电机初始化
- 编码器初始化
- VOFA 初始化
- 默认参数加载
- PID 初始化
- 当 `skipICM == 0` 时调用 `ICM42688_Init()`

之后它还会判断:

- `ICM42688_GetWhoAmI() == 0x85u`

如果成立, 就说明驱动认为当前已经成功识别到了 `BN0085`。

这一步之后还会继续做:

- `ICM42688_Calibrate(&sys->icm, 100)`
- `(void)ICM42688_ReadAll(&sys->icm)`
- `control_imu_zero_update(sys)`
- `ICM42688_ResetAttitude(&sys->icm)`

也就是说, 初始化阶段不仅要求设备能握手, 还要求后续数据结构进入一个可运行状态。

6. ICM42688_Init() 实际做了什么

虽然函数名叫 `ICM42688_Init()`, 但当前实现完全是在初始化 `BN0085`。

整个过程可以分成 6 个阶段。

6.1 配置 GPIO 与软件 I2C

函数先打开:

- `GPIOA`
- `GPIOB`
- `AFIO`

并关闭 JTAG 的一部分复用：

- `GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE)`

随后配置：

- `PB12/PB13` 为开漏输出，用作软件模拟 I2C
- `PB14` 为推挽输出，用作 `RST`
- `PA15` 为上拉输入，用作 `INT`

然后默认拉高：

- `RST`
- `SCL`
- `SDA`

这说明总线与复位脚都会被驱动到一个已知的初始状态。

6.2 清空运行时状态

驱动会清空一系列内部变量，例如：

- 包序号 `g_bnoSeq`
- 接收头 `g_bnoHeader`
- 数据缓冲 `g_bnoData`
- `g_bnoYawBase`
- `g_bnoRawYaw`
- `g_bnoReady`
- `g_bnoInitStage`
- `g_bnoReadFailStreak`
- 最近通道、最近 report id、最近 payload 长度等诊断字段

这一步保证每次初始化都是“从干净状态开始”。

6.3 轮流尝试多个地址

`ICM42688_Init()` 会用一个地址表轮流调用：

- `BN0085_AttemptInitAtAddr(0x4A, ...)`
- `BN0085_AttemptInitAtAddr(0x4B, ...)`

- `BN0085_AttemptInitAtAddr(0x28, ...)`
- `BN0085_AttemptInitAtAddr(0x29, ...)`

如果任一地址初始化成功，就停止继续尝试。

6.4 如果没成功，再扫描总线

如果前面的固定地址都没成功，驱动还会执行：

- `BN0085_ScanBus()`

它会扫 `0x08 ~ 0x77` 全范围地址，记录：

- 第一个命中的地址
- 最后一个命中的地址
- 总命中数

如果扫描到设备，再尝试对扫描到的地址做初始化。

6.5 如果仍失败，再带硬复位重试一轮

如果不带复位的尝试仍失败，驱动会再来一轮：

- `BN0085_AttemptInitAtAddr(addr, 1u, ...)`

其中 `doReset = 1` 会触发：

- `BN0085_HardwareReset()`

也就是：

- `RST` 拉低 10ms
- 再拉高
- 等待设备重新启动

6.6 成功后退出初始化忙状态

不论成功还是失败，最后都会把：

- `g_bnoInitBusy = 0`

清掉，允许后续运行期读包或自动恢复流程继续工作。

7. BNO085 初始化内部的关键握手流程

每次 `BNO085_AttemptInitAtAddr()` 都会按下面顺序做握手。

7.1 确认设备存在

先调用：

- `BNO085_WaitPresent()`
- `BNO085_WaitPresentStable()`

底层依赖的是：

- `BNO085_DevicePresent()`

它本质上就是往当前 I2C 地址发地址字节，看设备是否 ACK。

因此这里验证的是：

- 总线通不通
- 当前地址有没有设备应答
- 设备应答是否稳定

这只是“设备在线”判据，还不是“姿态数据已经能读”。

7.2 请求 Product ID

接着调用：

- `BNO085_RequestProductId()`

该函数会通过 `SHTP` 控制通道发送：

- `BNO085_REPORT_PRODUCT_ID_REQUEST (0xF9)`

如果后续收到了：

- `BN0085_REPORT_PRODUCT_ID_RESPONSE (0xF8)`

就会把：

- `g_icmWho = 0x85u`

这就是为什么在上层控制代码里，仍然通过检查 `ICM42688_GetWhoAmI() == 0x85u` 来判断 `BN0085` 是否被成功识别。

7.3 开启需要的传感器报告

握手通过后，会发送 `Set Feature Command` 开启以下报告：

- `Rotation Vector`
- `Game Rotation Vector`
- `Gyroscope`
- `Accelerometer`

其中报告周期设置为：

- `BN0085_REPORT_INTERVAL_US = 20000`

也就是约 `20ms` 一次。

7.4 等待第一帧姿态报告

开启 feature 后，驱动会进入：

- `BN0085_WaitForFirstReport()`

这里并不是单纯等任意包，而是要等到 `ICM42688_ReadAll(data)` 真的把姿态报告解析出来，并且：

- `data->ahrsInited == 1`

只有满足这个条件，才认为初始化真正完成。

7.5 初始化成功的最终标志

当上面步骤全部走通后，会设置：

- `g_bnoReady = 1`
- `g_icmWho = 0x85u`
- `g_bnoInitStage = BNO085_INIT_STAGE_READY`

这意味着：

- 设备在线
- SHTP 握手成功
- feature 已开启
- 第一帧姿态报告已到达
- 工程已经具备持续读 yaw 的基础

8. 软件 I2C 与 SHTP 收发是怎么工作的

8.1 软件 I2C

当前工程没有直接使用 STM32 硬件 I2C 外设，而是在 `ICM42688.c` 里自己实现了一套 bit-bang I2C。

关键函数包括：

- `ICM_I2C_Start()`
- `ICM_I2C_Stop()`
- `ICM_I2C_SendByte()`
- `ICM_I2C_RecvByte()`
- `ICM_I2C_WaitAck()`
- `ICM_I2C_WaitSclHigh()`

这套实现依赖：

- `PB12/PB13` 的电平切换
- 开漏输出
- 适当的延时 `ICM_I2C_Delay()`

8.2 BNO085 的收发不是寄存器读写，而是包式协议

与普通寄存器型 IMU 不同，`BNO085` 在这里不是靠“读某个寄存器地址”来拿姿态，而是走 `SHTP` 包协议。

工程中对应函数为：

- `BNO085_SendPacket()`
- `BNO085_ReceivePacket()`
- `BNO085_ReadBlock()`
- `BNO085_WriteBytes()`

8.3 `INT` 脚参与了读包门控

当前 `BNO085_ReceivePacket()` 一开始就会检查：

- `INT` 是否为低电平

代码逻辑是：

- 如果 `INT` 不是低电平，直接返回 0
- 只有 `INT` 拉低时，才认为 `BNO085` 当前有数据可读

这说明当前驱动不是无脑轮询整个总线，而是用 `INT` 作为“数据就绪”门控信号。

这是当前工程能较稳定读包的关键点之一。

9. 收到包之后，如何从四元数算出 yaw

9.1 解析姿态报告

当 `ICM42688_ReadAll()` 被调用时，它会循环尝试收若干个包。

只要收到的包来自：

- `BNO085_CHANNEL_REPORTS`
- 或 `BNO085_CHANNEL_WAKE_REPORTS`

就会继续调用：

- `BNO085_ParseSensorReport(data, len)`

9.2 识别 report id

`BN0085_ParseSensorReport()` 会先检查:

- 包是否是 `BASE_TIMESTAMP` 格式
- 当前 `reportId` 是什么

支持解析的主要 report 包括:

- `Accelerometer`
- `Gyroscope`
- `Rotation Vector`
- `Game Rotation Vector`

其中真正决定姿态的, 是:

- `BN0085_SENSOR_ROTATION_VECTOR`
- `BN0085_SENSOR_GAME_ROTATION_VECTOR`

9.3 从 report 中取出四元数

当 report 是 rotation vector 时, 驱动会把数据解成:

- `q1`
- `q2`
- `q3`
- `q0`

并写入 `ICM42688_Data_t`。

随后执行:

- `data->ahrsInited = 1`
- `data->yawSampleUpdated = 1`
- `BN0085_UpdateEuler(data)`

这里 `ahrsInited` 表示:

- 已经收到过有效姿态四元数

而 `yawSampleUpdated` 表示:

- 这一次读包过程中，姿态样本刚刚被更新过

9.4 四元数转换成欧拉角

`BN0085_UpdateEuler()` 会基于四元数计算：

- `yaw`
- `pitch`
- `roll`

其中最关键的是：

- 先算原始偏航 `g_bnoRawYaw`
- 再减掉基准 `g_bnoYawBase`
- 最后做 `[-180, 180]` 包角处理

最终写入：

- `data->yaw`

所以当前工程里的偏航角本质上是：

- **BN0085 输出的四元数姿态**
- 经过 STM32 侧四元数转欧拉角
- 再减去 “最近一次复位姿态时记录下来的偏航零点”

得到的相对偏航角。

10. 为什么 `ICM42688_UpdateYaw()` 还要再更新一次 `yaw`

这一步很容易让人疑惑：

- 既然 `BN0085_UpdateEuler()` 已经算了 `yaw`
- 为什么后面还有 `ICM42688_UpdateYaw()`

原因是：

10.1 BN0085_UpdateEuler() 更像 “原始姿态解算结果落表”

它做的核心事是：

- 从四元数直接算出当前姿态角
- 让 `data->yaw/pitch/roll` 及时刷新

10.2 ICM42688_UpdateYaw() 做的是运行时保护与动态量计算

这个函数会进一步处理：

- 再次根据 `g_bnoRawYaw - g_bnoYawBase` 生成 `newYaw`
- 计算相邻样本之间的角度差 `dy`
- 根据 `dt` 估算 `yawRate`
- 对 `yawRate` 做限幅和低通
- 当跳变过大时，拒绝这次异常更新

里面的保护项包括：

- `BN0085_YAW_JUMP_REJECT_DEG = 45.0f`
- `BN0085_YAW_RATE_LIMIT_DPS = 180.0f`
- `BN0085_YAW_RATE_LPF_ALPHA = 0.35f`

因此 `ICM42688_UpdateYaw()` 的作用是：

- 给 `yaw` 增加运行期异常跳变抑制
- 给控制环提供更平滑可靠的 `yawRate`
- 提高航向环对坏样本的抗扰性

所以它不是重复计算，而是 “姿态落地后的二次运行期处理”。

11. 主循环中，yaw 是在什么时候持续刷新的

11.1 Control_Tick() 每 1ms 走一次主逻辑节拍

在 `User/main.c` 主循环中，程序会不断消费 `g_tim4PendingTicks`，每消费一次就调用：

- `Control_Tick(&g_controlSys)`

这意味着：

- `tickCount` 是按系统节拍不断推进的

11.2 IMU 更新不是每次都读，而是每 5ms 读一次

在 `Control_Tick()` 内部有这段逻辑：

- 当 `sys->tickCount % 5 == 0` 时
- 如果当前是 `TEST_MODE_NORMAL`
- 就调用 `control_imu_update(sys)`

因此当前 `yaw` 的主循环刷新节拍约为：

- 5ms 一次

11.3 control_imu_update() 是 yaw 刷新的主入口

这个函数里做了三件关键事：

1. 调 `ICM42688_ReadAll(&sys->icm)` 从 BNO085 收包并更新结构体
2. 如果 `yawSampleUpdated` 为真，则计算 `dt`，再调 `ICM42688_UpdateYaw()`
3. 根据 `targetYaw` 和 `currentYaw` 计算 `yawErr`，再生成 `headingCorr`

也就是说，真正让 `sys->icm.yaw` 在运行时不断变化的核心组合是：

- `ICM42688_ReadAll()`
- `BNO085_ParseSensorReport()`
- `BNO085_UpdateEuler()`

- `ICM42688_UpdateYaw()`
-

12. yaw 是如何进入控制环的

当前工程不是“只把 yaw 显示出来”，而是直接把它接进航向控制。

12.1 锁定目标航向

`Control_LockHeading(sys)` 的逻辑非常简单：

- `sys->targetYaw = sys->icm.yaw`

也就是说，锁航时不会写死某个绝对方位，而是：

- 把当前朝向当成目标朝向

12.2 启动时重新建立 yaw 零点

在 `Control_Start()` 中，系统会：

- 先等待 IMU ready
- 再 `control_imu_zero_update(sys)`
- 再 `ICM42688_ResetAttitude(&sys->icm)`
- 然后把 `sys->targetYaw = sys->icm.yaw`
- 再设 `headingLockPending = 1`

而 `ICM42688_ResetAttitude()` 会把：

- `g_bnoYawBase = g_bnoRawYaw`
- `data->yaw = 0`
- `data->prevYaw = 0`
- `data->yawRate = 0`

这意味着每次重新启动时，系统都会把“当前朝向”重新当成新的相对零点。

12.3 计算偏航误差

`control_imu_update()` 在每次刷新后都会调用：

- `ICM42688_GetYawError(sys->targetYaw, sys->icm.yaw)`

它会把误差包到 `[-180, 180]`，避免跨 $\pm 180^\circ$ 时出现大跳变。

最终得到：

- `sys->yawErr`

12.4 生成 heading correction

随后 `Control.c` 根据：

- `hp * yawErr`
- `hi * 积分`
- `hd * yawRate`

生成：

- `sys->headingCorr`

这一步还带有：

- 死区
- 启动初期冻结
- 启动初期限幅
- 启动初期缓升
- 输出总限幅

因此 `yaw` 在控制环中的作用不是装饰性的，而是直接决定左右轮差速修正量。

12.5 混入左右轮输出

在输出阶段，`headingCorr` 会被量化成差分修正后混到：

- `outLQ += diffQ`
- `outRQ -= diffQ`

最后通过：

- `Motor_SetDiffSpeed(outL, outR)`

输出到电机。

所以完整链路是：

- BN0085 -> q -> yaw -> yawErr -> headingCorr -> 左右轮差速
-

13. 工程是靠什么判定 “已经成功读到偏航角” 的

成功读到 yaw 不能只看一个条件，而要分层判断。

13.1 第一层：设备已识别

判据：

- ICM42688_GetWhoAmI() == 0x85u

说明：

- Product ID 请求已经拿到响应
- 至少初始化链路走通了一大半

13.2 第二层：初始化阶段已进入 READY

判据：

- ICM42688_GetInitStage() == 6

即：

- BN0085_INIT_STAGE_READY

说明：

- 设备在线
- 已打开 feature
- 已等到第一帧可用姿态报告

13.3 第三层：姿态样本已建立

判据：

- `sys->icm.ahrsInitd == 1`

说明：

- 至少已经成功解析过一次 rotation vector

13.4 第四层：运行期 yaw 在刷新

判据：

- `sys->icm.yaw` 不再固定为 0
- 板子转动时 yaw 能跟着变化
- `yawRate` 也会出现对应变化

这才算真正意义上的“已经成功读到偏航角并在运行中使用”。

14. 当前工程有哪些现成调试手段可以验证 yaw 读取成功

14.1 OLED 主界面

主界面会显示：

- yaw
- yawErr
- headingCorr
- 当前 who / addr / stage / probeAddr

其中最关键的是第 4 行里这些字段：

- Wxx：当前识别结果
- Axx：当前 I2C 地址
- Sx：初始化阶段

- Pxx: 最近探测地址

如果看到:

- W85
- S6

说明初始化已经完全进入 ready 状态。

14.2 OLED 引脚/ACK 诊断页

main.c 里还有 pin 菜单, 可以翻页查看:

- SCL/SDA 当前电平
- RST/INT 当前电平
- 地址 0x28/0x29/0x4A/0x4B 是否 ACK

这有助于判断问题发生在:

- 总线层
- 地址层
- 复位层
- 数据层

14.3 VOFA 文本调试输出

BN0085_DebugSerial_Send() 会定期发送一整行调试信息, 内容包括:

- y: yaw
- yr: yawRate
- q0/q1/q2/q3
- who
- addr
- paddr
- st
- pwho
- sfa/sla/shc
- ch/rid/len
- txch/txlen/wfail

- `scl/sda/rst/int`
- `iok/if`
- `rx/txd`

这是当前工程判断“到底成功到了哪一步”的最强诊断接口。

14.4 串口命令 `#IMU_INIT!`

`Control.c` 里保留了：

- `#IMU_INIT!`

它可以在线重新执行：

- `ICM42688_Init()`

方便你在不重启系统的情况下重做初始化和抓日志。

15. 运行期失败后，工程是如何自动恢复的

当前工程不是“只初始化一次，失败就彻底废掉”，而是有恢复策略。

15.1 未 ready 长时间失败，会重初始化

在 `ICM42688_ReadAll()` 中：

- 如果 `g_bnoReady == 0`
- 且连续失败达到 `BN0085_RECOVER_NOT_READY_FAILS`

就会调用：

- `BN0085_TryRecover(data)`

15.2 已 ready 但运行期卡死，也会重初始化

如果原本已经 ready，但之后长期没有解析到新包，也会累计：

- `g_bnoReadFailStreak`

超过 `BN0085_RECOVER_STALL_FAILS` 后，会执行：

- 清 `g_bnoReady`
- 清 `g_icmWho`
- 把阶段退回 `probe`
- 调 `BN0085_TryRecover(data)`

15.3 恢复时会把姿态状态清零

`BN0085_TryRecover()` 会把这些状态清掉：

- `ahrsInitd`
- `yaw`
- `yawRate`
- `prevYaw`
- `yawRateValid`
- `yawSampleUpdated`

然后重新走 `ICM42688_Init()`。

这避免了旧姿态状态污染新一轮初始化结果。

16. 为什么这个工程最终能够 “成功读到 yaw”

把前面所有环节串起来，当前工程之所以能够成功读到偏航角，本质上依赖以下几个关键设计同时成立。

16.1 已经不再把 BNO085 当成寄存器型 IMU 使用

而是正确按它的包协议去收发：

- 用 `SHTP` 发控制命令
- 用 `SHTP` 收姿态报告

这是从“能不能读到 BNO085”角度最根本的一步。

16.2 初始化过程不是单点硬编码，而是有多轮兜底

初始化链路包含：

- 多地址尝试
- 全总线扫描
- 不带复位尝试
- 带硬复位重试
- 等待设备稳定出现
- 等待第一帧姿态包

这使得工程对模块差异和上电时序差异的容忍度明显提高。

16.3 收包时利用了 INT 做数据门控

INT 为低才读包，避免了很多无意义空读。

这使得驱动更容易在正确时机取到完整 SHTP 包。

16.4 不是只拿四元数，还做了姿态后处理

BNO085_UpdateEuler() + ICM42688_UpdateYaw() 组合带来了：

- 相对零点
- 包角处理
- 跳变拒绝
- yawRate 低通

这保证了输出给控制环的 yaw 和 yawRate 更适合闭环控制，而不是仅仅“能显示”。

16.5 启动流程会重新锁定当前朝向

Control_Start() 中会：

- 等 IMU ready
- reset attitude
- 以当前朝向作为新零点与目标航向

因此每次启动时，控制系统都能从“当前姿态”开始锁航，不会继承上一次运行残留的全局角度偏移。

17. 一句话总结这条 yaw 链路

可以把当前 `Project_Refactor` 中的偏航角读取链路总结为下面这一句话：

STM32 通过 PB12/PB13 软件 I2C 与 BNO085 通信，利用 PB14 复位、PA15 作为数据就绪门控，先通过 SHTP 完成 Product ID 握手和 feature 开启，再持续接收 Rotation Vector / Game Rotation Vector 报告，把四元数转换成相对 yaw，并在 Control 层计算 yawErr 与 headingCorr，最终用于左右轮差速控制。

18. 简化版调用链总览

为了便于以后快速回忆，这里给出一份简化版调用链：

18.1 上电初始化链路

```
main() -> Control_Init(&g_controlSys, 0) -> ICM42688_Init() ->
BNO085_AttemptInitAtAddr() -> BNO085_WaitPresent() ->
BNO085_RequestProductId() -> BNO085_EnableReport() ->
BNO085_WaitForFirstReport() -> g_bnoReady = 1
```

18.2 运行期读取链路

```
Control_Tick() -> 每 5ms 调 control_imu_update() ->
ICM42688_ReadAll(&sys->icm) -> BNO085_ReceivePacket() ->
BNO085_ParseSensorReport() -> BNO085_UpdateEuler() ->
ICM42688_UpdateYaw()
```

18.3 控制链路

```
sys->icm.yaw -> ICM42688_GetYawError(targetYaw, currentYaw) -> sys->yawErr -> sys->headingCorr -> 混入左右轮 PWM -> Motor_SetDiffSpeed()
```

19. 对后续阅读源码的建议

如果后面你还要继续改这套 IMU 链路，建议按下面顺序读源码：

1. 先读 `User/main.c`
2. 再读 `Hardware/Control.c` 中的
 - `Control_Init()`
 - `Control_Start()`
 - `Control_Tick()`
 - `control_imu_update()`
3. 再读 `Hardware/ICM42688.c` 中的
 - `ICM42688_Init()`
 - `BN0085_AttemptInitAtAddr()`
 - `BN0085_RequestProductId()`
 - `BN0085_EnableReport()`
 - `ICM42688_ReadAll()`
 - `BN0085_ParseSensorReport()`
 - `BN0085_UpdateEuler()`
 - `ICM42688_UpdateYaw()`
4. 最后用 `Hardware/BN0085_DebugSerial.c` 对照调试输出理解运行状态

按这个顺序看，最容易把“初始化成功”和“运行时成功更新 yaw”这两件事分开看清楚。

20. 本文结论

当前 `Project_Refactor` 之所以能成功读到偏航角，并不是因为 STM32 自己完成了完整姿态融合，而是因为：

- 底层已经把 `BN0085` 正确接入到当前工程

- 通过软件 I2C + SHTP 包协议完成了初始化和读包
- 通过 rotation vector 拿到了四元数
- 再在 STM32 侧转换成 yaw
- 最后把 yaw 接入了航向锁定和差速控制

换句话说，当前工程中“成功读到偏航角”的关键不是单一某个公式，而是下面整条链路都打通了：

- **硬件引脚正确**
- **I2C 地址可达**
- **SHTP 握手成功**
- **feature 开启成功**
- **第一帧姿态报告到达**
- **四元数解析成功**
- **yaw 在主循环持续刷新**
- **yaw 被控制环实际使用**

只要其中某一环断掉，就会表现成“读不到 yaw”或者“yaw 不更新”。